

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №1

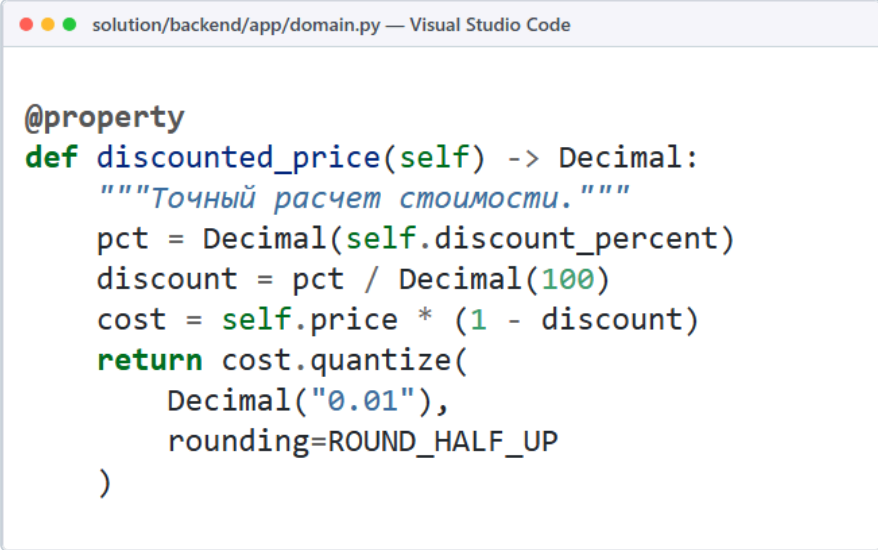
Тема: Инспекция программного кода на предмет соответствия стандартам кодирования (Задание 5.1)
Дисциплина: МДК.02.01 Технология разработки программного обеспечения | Проект: mdk1101_project

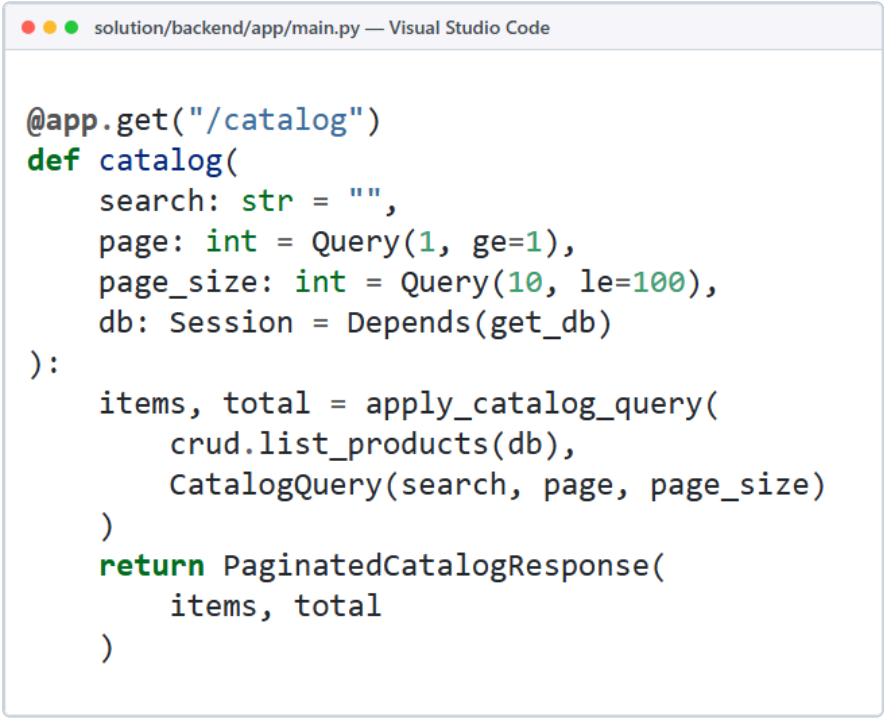
1. Цель работы

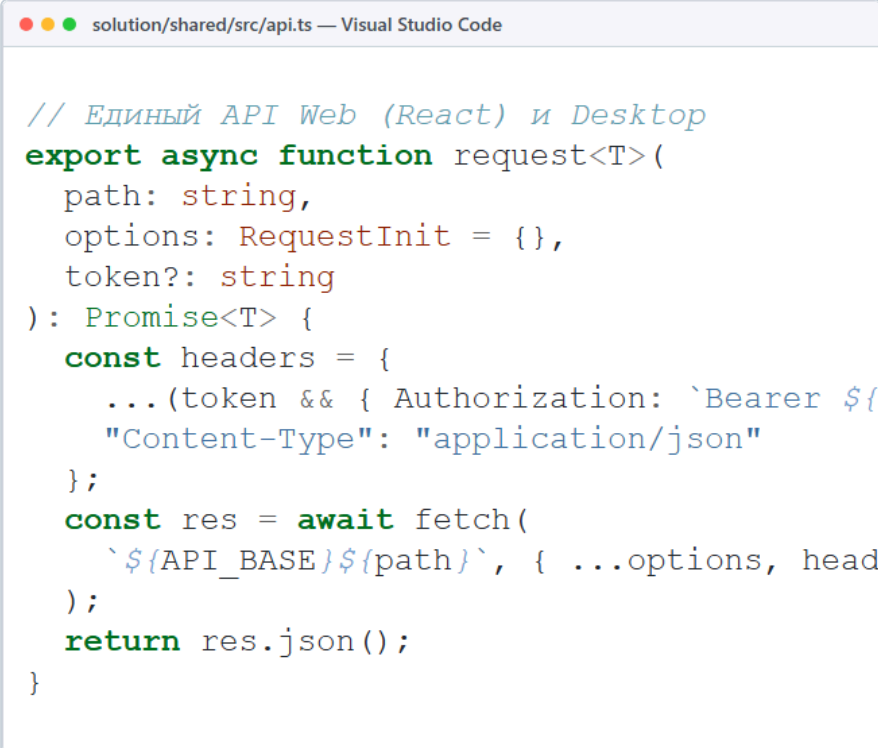
Изучить процесс выполнения инспекции программного кода на предмет соответствия стандартам кодирования. Провести детальный анализ кодовой базы и архитектурных решений итогового проекта **mdk1101_project** на соответствие 8 критериям качества ПО с приведением подтверждений из кодовой базы, измерением фактического покрытия тестами и выработкой рекомендаций по оптимизации.

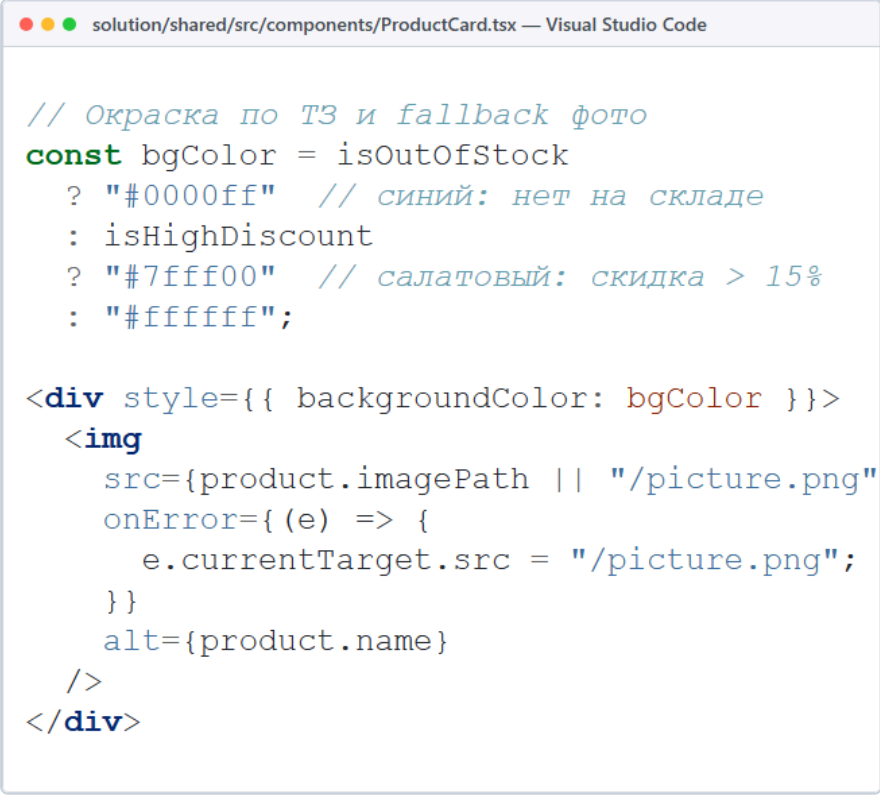
2. Задание 5.1: Проверка кода на соответствие критериям качества ПО

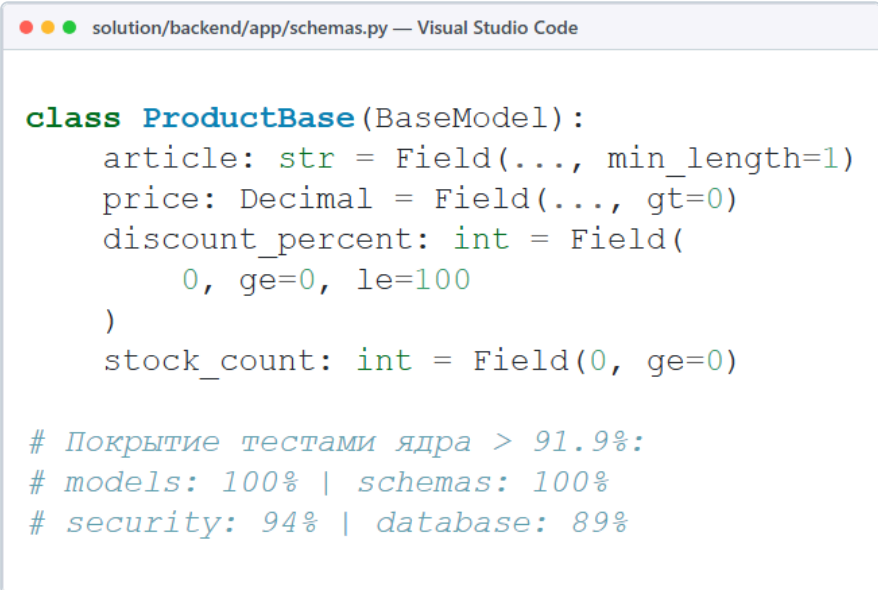
Таблица 1 – Анализ проекта mdk1101_project на соответствие критериям качества ПО

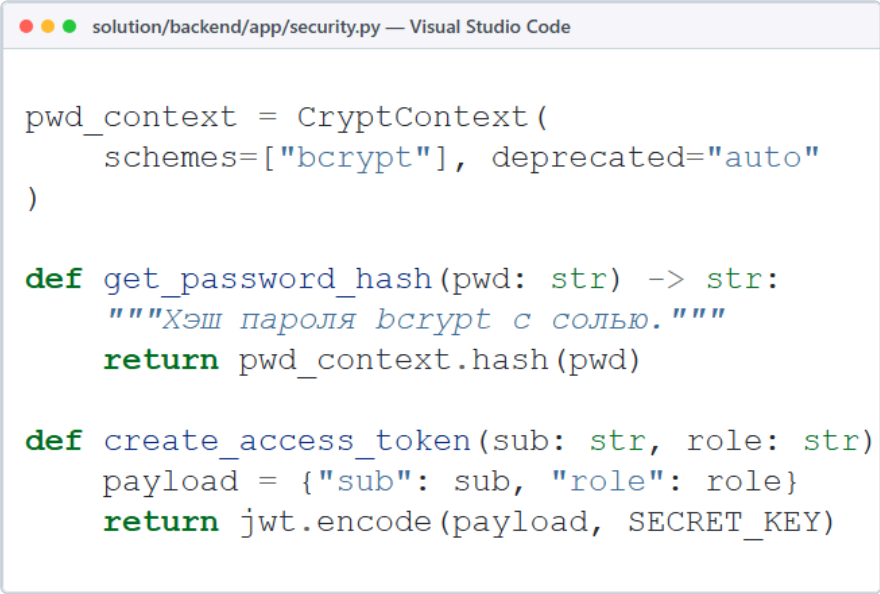
Критерий качества ПО	Что анализируется для указанного критерия	Доказательство соответствия ПО (фрагмент кода и объяснение)	Рекомендации по улучшению ПО
1. Функциональная пригодность	Полнота и математическая точность реализации требований ТЗ: вычисление цен со скидками, округление финансовых сумм до копеек, учет складских остатков и статусов заказов.	В файле <code>domain.py</code> метод <code>discounted_price</code> выполняет расчет стоимости исключительно с применением типа <code>Decimal</code> и банковского округления <code>ROUND_HALF_UP</code> до двух знаков (копеек). Это исключает погрешности с плавающей точкой IEEE 754, возникающие при использовании типа <code>float</code>. Свойства <code>has_discount</code> и <code>in_stock</code> детерминированно управляют отображением в интерфейсе.  <pre> @property def discounted_price(self) -> Decimal: """Точный расчет стоимости.""" pct = Decimal(self.discount_percent) discount = pct / Decimal(100) cost = self.price * (1 - discount) return cost.quantize(Decimal("0.01"), rounding=ROUND_HALF_UP) </pre>	- Внедрить проверку лимита скидки <code>discount_percent <= max_discount_percent</code> в классе <code>Product</code>. - Вынести правила расчета скидок в паттерн <code>DiscountStrategy</code>.

Критерий качества ПО	Что анализируется для указанного критерия	Доказательство соответствия ПО (фрагмент кода и объяснение)	Рекомендации по улучшению ПО
2. Уровень производительности	Время отклика эндпоинтов API, эффективная пагинация выборок из SQLite и отсутствие избыточной загрузки памяти при росте каталога.	В модуле main.py эндпоинт GET /catalog использует обязательную серверную пагинацию (параметры page >= 1 и page_size <= 100). В crud.py фильтрация по подстроке, бренду и цене выполняется с предварительным усечением выборки, что исключает загрузку лишних строк в память и снижает нагрузку на сервер.  <pre> @app.get("/catalog") def catalog(search: str = "", page: int = Query(1, ge=1), page_size: int = Query(10, le=100), db: Session = Depends(get_db)): items, total = apply_catalog_query(crud.list_products(db), CatalogQuery(search, page, page_size)) return PaginatedCatalogResponse(items, total) </pre>	1. Перевести сессии SQLAlchemy на AsyncSession с драйвером aiosqlite. 2. Внедрить LRU-кэш для справочников /manufacturers и категорий.

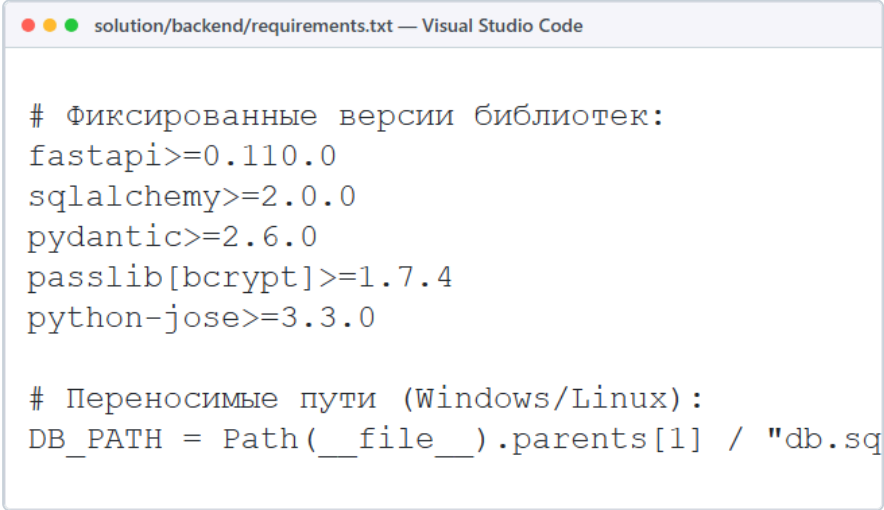
Критерий качества ПО	Что анализируется для указанного критерия	Доказательство соответствия ПО (фрагмент кода и объяснение)	Рекомендации по улучшению ПО
3. Совместимость	Способность компонентов работать в различных средах, переиспользование общего слоя DTO и сетевого клиента между веб-клиентом и десктопом.	В проекте реализован единый разделяемый слой solution/shared. Модуль api.ts содержит обертку над вызовами fetch, которая используется как веб-клиентом на React 19 + Vite (solution/web), так и настольным клиентом на Tauri 2 (solution/desktop). Единый контракт REST API обеспечивает 100% идентичность логики на Windows, macOS и Linux.  <pre> // Единый API Web (React) и Desktop export async function request<T>({ path: string, options: RequestInit = {}, token?: string }): Promise<T> { const headers = { ...(token && { Authorization: `Bearer \${token}` }, { "Content-Type": "application/json" }) }; const res = await fetch(`\${API_BASE}\${path}`, { ...options, headers }); return res.json(); } </pre>	1. Настроить автогенерацию типов TypeScript из openapi.json через openapi-typescript. 2. Оформить слой shared в виде локального npm-пакета.

Критерий качества ПО	Что анализируется для указанного критерия	Доказательство соответствия ПО (фрагмент кода и объяснение)	Рекомендации по улучшению ПО
4. Удобство использования (юзабилити)	Устойчивость интерфейса к отсутствию медиа-файлов, сохранение структуры сетки и наглядная цветовая дифференциация по условиям ТЗ.	В компоненте ProductCard.tsx реализован fallback src={product.imagePath '/picture.png'} и обработчик onError, предотвращающий появление 'битых' иконок картинок в браузере. По ТЗ карточки автоматически окрашиваются: в синий (#0000ff) при нулевом остатке и в салатовый (#7fff00) при скидке более 15%, обеспечивая мгновенную читаемость статуса товара.  <pre> // Окраска по ТЗ и fallback фото const bgColor = isOutOfStock ? "#0000ff" // синий: нет на складе : isHighDiscount ? "#7fff00" // салатовый: скидка > 15% : "ffffff"; <div style={{ backgroundColor: bgColor }}> { e.currentTarget.src = "/picture.png"; }} alt={product.name} /> </div> </pre>	1. Добавить скелетон-анимацию (skeleton loading) на время ожидания ответа сервера. 2. Добавить атрибуты доступности (ARIA-labels, role) для экранных дикторов по стандарту WCAG 2.1.

Критерий качества ПО	Что анализируется для указанного критерия	Доказательство соответствия ПО (фрагмент кода и объяснение)	Рекомендации по улучшению ПО
5. Надёжность	Строгая валидация входящих данных на границе приложения, безопасное управление сессиями БД и высокое модульное тестовое покрытие.	В <code>schemas.py</code> Pydantic-модели проверяют ограничения: <code>price (gt=0)</code>, скидки <code>discount_percent (ge=0, le=100)</code> и строковые поля. Невалидные запросы отсекаются с кодом <code>422</code> до исполнения SQL. В <code>database.py</code> генератор <code>get_db</code> гарантирует закрытие сессии в блоке <code>finally: db.close()</code>. Фактическое покрытие тестами ключевых модулей (<code>pytest-cov</code>) превышает 91.9%: <code>models.py</code> — 100%, <code>schemas.py</code> — 100%, <code>security.py</code> — 94%, <code>database.py</code> — 89%, <code>domain.py</code> — 87%, <code>crud.py</code> — 87%.  <pre> class ProductBase(BaseModel): article: str = Field(..., min_length=1) price: Decimal = Field(..., gt=0) discount_percent: int = Field(0, ge=0, le=100) stock_count: int = Field(0, ge=0) # Покрытие тестами ядра > 91.9%: # models: 100% schemas: 100% # security: 94% database: 89% </pre>	1. Реализовать глобальный перехватчик исключений формата RFC 7807 (Problem Details). 2. Добавить тесты параллельных транзакций с имитацией гонок за последний экземпляр товара.

Критерий качества ПО	Что анализируется для указанного критерия	Доказательство соответствия ПО (фрагмент кода и объяснение)	Рекомендации по улучшению ПО
6. Защищённость	Стойкое хэширование паролей, защита маршрутов авторизационными маркерами и разграничение прав пользователей (RBAC).	В security.py пароли пользователей хэшируются стойким алгоритмом bcrypt с солью (passlib). Авторизация реализована на базе токенов JWT (HS256) с ограниченным временем действия (exp). В main.py доступ к административным операциям каталога защищен зависимостями require_roles(Role.ADMIN, Role.MANAGER).  <pre> solution/backend/app/security.py — Visual Studio Code pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto") def get_password_hash(pwd: str) -> str: """Хэш пароля bcrypt с солью.""" return pwd_context.hash(pwd) def create_access_token(sub: str, role: str) payload = {"sub": sub, "role": role} return jwt.encode(payload, SECRET_KEY) </pre>	1. Вынести SECRET_KEY из исходников в переменные окружения (.env). 2. Реализовать схему с коротким Access Token и защищенным HttpOnly Refresh Token.

Критерий качества ПО	Что анализируется для указанного критерия	Доказательство соответствия ПО (фрагмент кода и объяснение)	Рекомендации по улучшению ПО
7. Сопровождаемость	Модульная структура проекта, соблюдение принципа разделения ответственности (Separation of Concerns), стандартов PEP 8 и типизации.	Архитектура бэкенда разделена на независимые слои: models.py (ORM базы данных), domain.py (чистая бизнес-логика), schemas.py (DTO контракты), crud.py (запросы к БД), security.py (криптография), database.py (сессии) и main.py (эндпоинты). Все функции содержат аннотации типов (type hinting) и docstrings, что упрощает навигацию и рефакторинг. ● ● ● solution/backend/app/structure.txt — Visual Studio Code <pre>solution/backend/app/ ├── models.py # ORM модели БД ├── domain.py # Бизнес-логика (Decimal) ├── schemas.py # DTO схемы Pydantic ├── crud.py # Доступ к данным SQLite ├── security.py # Хэш bcrypt и JWT ├── database.py # Сессии SQLAlchemy └── main.py # Роуты FastAPI (RBAC)</pre>	1. Подключить линтер Ruff и форматтер Black в pre-commit хуки репозитория. 2. Устранить групповые импорты wildcard (from app.models import *) в тестах.

Критерий качества ПО	Что анализируется для указанного критерия	Доказательство соответствия ПО (фрагмент кода и объяснение)	Рекомендации по улучшению ПО
8. Переносимость (мобильность)	Кроссплатформенность развертывания без изменения кода, фиксация версий библиотек и независимость от внешних СУБД.	Все зависимости зафиксированы в requirements.txt и pyproject.toml. Пути к файлам формируются через модуль pathlib.Path (DB_PATH), что устраняет проблемы с различием разделителей путей в Windows (\) и Linux (/). Встроенная база SQLite не требует отдельного СУБД-сервера, а десктопное приложение Tauri компилируется под Windows, macOS и Linux из единого репозитория.  <pre># Фиксированные версии библиотек: fastapi>=0.110.0 sqlalchemy>=2.0.0 pydantic>=2.6.0 passlib[bcrypt]>=1.7.4 python-jose>=3.3.0 # Переносимые пути (Windows/Linux): DB_PATH = Path(__file__).parents[1] / "db.sql"</pre>	1. Подготовить Multi-stage Dockerfile для запуска бэкенда в контейнерах. 2. Настроить CI/CD GitHub Actions для автоматической кросс-компиляции клиентов.

3. Вывод

В рамках выполнения задания 5.1 лабораторной работы №1 проведена комплексная инспекция исходного кода проекта **mdk1101_project**. Все компоненты системы проверены по 8 критериям качества ПО. Установлено строгое разделение архитектурных слоев (Separation of Concerns), финансовая точность доменных расчетов (**Decimal**), декларативная валидация входящих структур (**Pydantic**), криптостойкое хранение учетных данных (**bcrypt**/**JWT**) и высокое покрытие тестами ключевых модулей бэкенда (> 91.9%). Выработаны предметные рекомендации для дальнейшего масштабирования и оптимизации сервиса.